

PROJECT AND TEAM INFORMATION

Project Title

Smart Attendance System Using Face Recognition

Student / Team Information

Team Name:

Byte Storm

Team member 1(Leader)

(Negi : Antriksh : 241601259:
antrikshnegi2007@gmail.com)



Team member 2

(Thapa, Kriti : 241602072 : kritithapa609@gmail.com)



Kriti Thapa
11/09/26.

Team member 3

(Tiwari : Ayush : 241601311 :
ayush.tiwari0618@gmail.com)



Team member 4:

(Chauhan : Rohit : 241601103 :
Rc9634708@gmail.com)



PROPOSAL DESCRIPTION

Motivation

Taking attendance manually in a typical college setting is surprisingly inefficient. In a standard 50-minute lecture, calling out names or passing around a sign-in sheet eats up 10 to 12 minutes of active teaching time. Paper registers are easily lost, damaged, or altered, making them a unreliable long-term tracking tool. Beyond simple time loss, traditional methods are prone to proxy attendance—students routinely sign in for absent peers or swap physical ID cards. Institutions have tried modern alternatives like RFID card scanners and dynamic QR codes, but these solutions carry their own design flaws. RFID cards can be handed off to friends, while dynamic QR codes displayed on a projector are quickly screenshotted and shared across group chats like WhatsApp. This makes digital roll calls just as easy to game as paper-based ones. To fix this, our team is building an automated attendance pipeline powered by computer vision. By replacing physical tokens with facial biometrics, a camera mounted at the classroom entrance or front wall can detect faces, extract geometric embeddings, match identities against registered profiles, and log attendance directly to a central database. However, traditional face recognition systems introduce a major security vulnerability: presentation spoofing. Most basic scripts cannot distinguish between a live human face and a high-resolution photo or video loop displayed on a smartphone screen. To solve this, our pipeline includes a dedicated Presentation Attack Detection (PAD) and liveness filtering module that intercepts video frames before they ever reach the recognition model. The system analyzes image crops in real time for screen glare, pixel grid artifacts, and unnatural paper reflections. If a crop fails this liveness check, it is discarded immediately. This saves compute resources by avoiding unnecessary embedding calculations while completely blocking proxy attendance attempts. Finally, a session-level deduplication layer prevents the system from spamming the database with duplicate entries for the same student during a single lecture. Our project solves this by deploying a lightweight Presentation Attack Detection (PAD) and liveness filter before the facial recognition engine. By inspecting incoming image crops for screen reflections, artificial pixel grids, and paper textures prior to computing vector embeddings, fake attempts are immediately rejected. This design prevents proxy attendance, protects the integrity of academic logs, and minimizes unnecessary computational load on local edge hardware.

State of the Art / Current solution

Traditional attendance tracking mechanisms—including paper sign-in sheets, RFID card taps, and dynamic QR check-ins—share a core security flaw: they authenticate physical tokens or credentials rather than the individual person. This design gap makes it trivial for absent individuals to be fraudulently marked present by peers. Computer vision systems address this issue by identifying unique biological characteristics. However, many existing open-source and commercial facial recognition workflows run deep feature extraction models and calculate distance metrics on every detected face without verifying physical presence first. This approach presents two major limitations:

1. **Spoofing Vulnerabilities:** Standard pipelines fail to differentiate between a live individual and a high-resolution photo or pre-recorded video displayed on a mobile screen, allowing fraudulent check-ins.
2. **Computational Inefficiency:** Generating high-dimensional vector embeddings for every bounding box—including non-human background artifacts or spoof attempts—wastes processor cycles and leads to severe latency on non-GPU hardware.

Our proposed system restructures this processing sequence by inserting a fast, texture-based liveness gatekeeper upstream. Raw video frames must pass physical authenticity checks before feature extraction models are invoked. By filtering out spoof attempts early, our pipeline ensures attendance data remains tamper-proof while keeping computational demands low enough to run efficiently on standard classroom hardware.

Project Goals and Milestones

Our objective is to build an anti-spoofing automatic attendance system that will work on lab hardware.

Phase 1 – Pipeline Flow & Requirements: Identify locations of the camera, frame rates, desired inference latencies and database table relations.

Phase 2 – Database: Build SQLite/PostgreSQL database to store student roll numbers, section metadata, reference facial embeddings and attendance log indexed by the date.

Phase 3 – Enrollment Workflow: Develop script to collect multiple facial angles of each student, calculate mean embeddings and store reference vectors.

Phase 4 – Video Detection: Set up OpenCV to pull video stream from USB/RTSP camera, dynamically scale down frames to extract faces under various lighting conditions.

Phase 5 – Liveness Verification Engine: Develop Presentation Attack Detection model to detect 2D prints and mobile screen replay prior to passing image further.

Phase 6 – Biometric Recognition: Integrate deep face-embedding model and match obtained 1D vector with the enrolled records using cosine similarity thresholds.

Phase 7 – Attendance Logic & Session Locking: Develop duplicate filtering mechanism to ensure that student will be logged only once within an active lecture period.

Phase 8 – Faculty Portal & Testing: Develop web portal (Flask/Streamlit) for professors to start sessions, view the logs in real-time, export attendance sheets in CSV format and test the system calculating false acceptance rate.

Major Milestones

Milestone 1 – Requirement Analysis: Identify issues associated with attendance, define functional and security requirements, choose required technologies and design the system workflow.

Milestone 2 – Database Development: Design databases of students and attendance and identify relationships between registered students and attendance records.

Milestone 3 – Student Registration: Develop student registration functionality and collect the required facial data for recognition.

Milestone 4 – Face Detection: Implement camera integration and face detection in real-time using OpenCV or another detection module.

Milestone 5 – Face Recognition: Integrate and test suitable face-recognition/deep-learning model.

Milestone 6 – Liveness Detection: Integrate and test anti-spoofing/liveness-detection model.

Milestone 7 – Attendance Processing: Connect recognition and liveness verification with attendance database and develop duplicate prevention.

Milestone 8 – Dashboard and Testing: Develop faculty/admin portal, perform functional and security testing, improve recognition performance and finalize project presentation. The last milestone will include testing in classroom-like environment and documentation of the system.

Project Approach

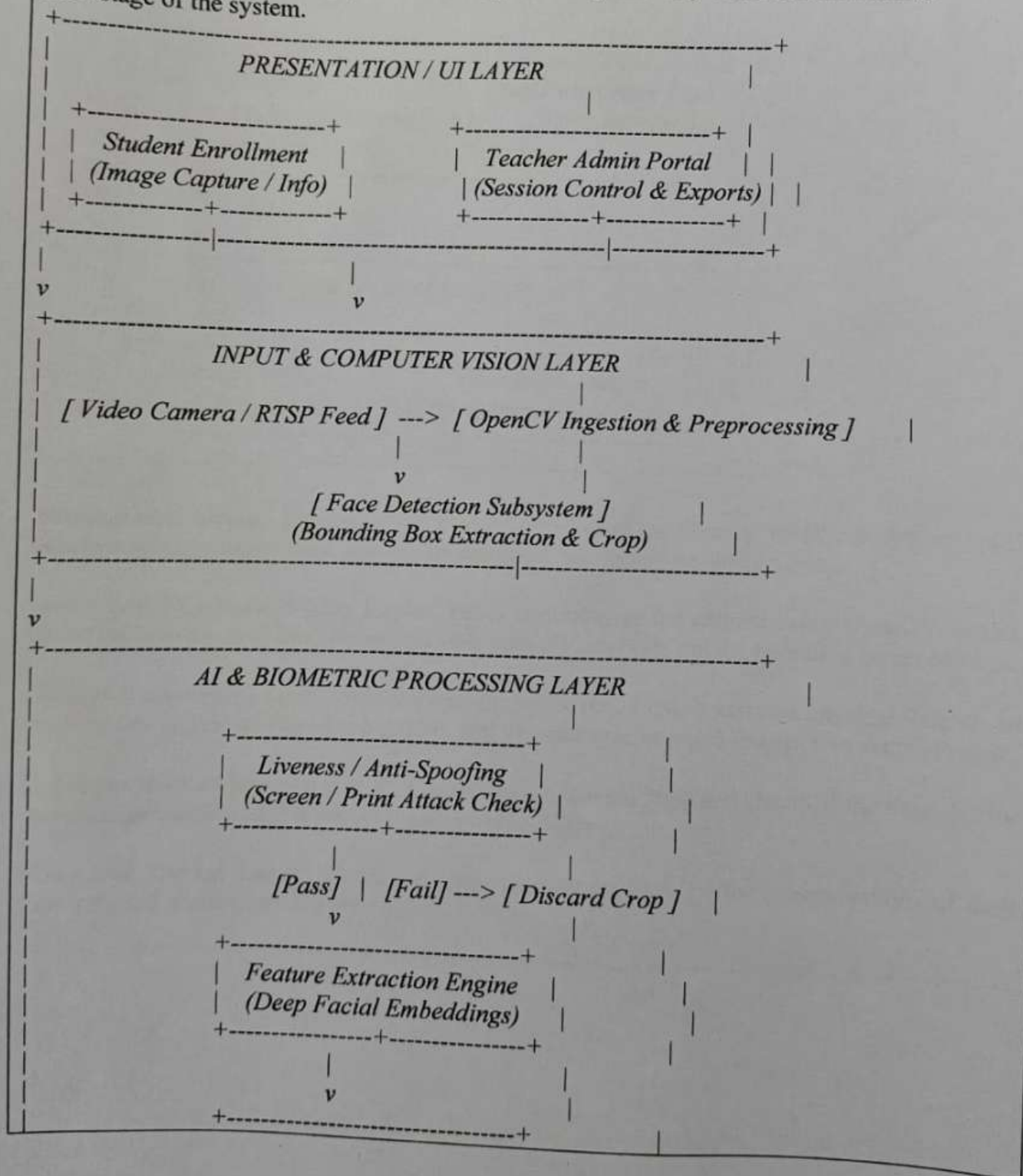
The system runs as an end-to-end Python pipeline structured to minimize inference latency on non-GPU workstations:

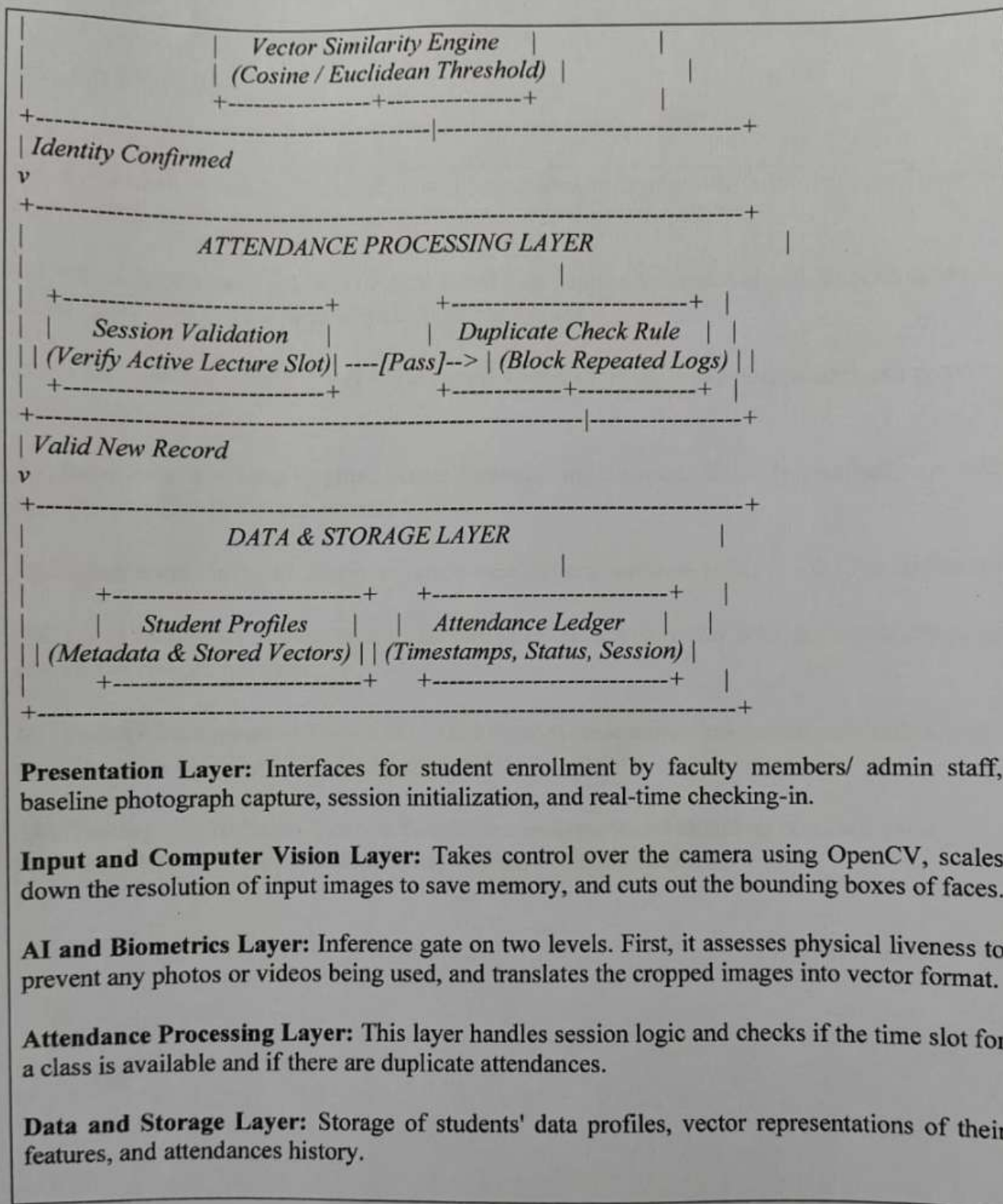
1. **Frame Capture & Normalization:** OpenCV grabs frames from a connected camera feed. Frames are downsampled to 720p to maintain frame rates, and basic histogram equalization is applied to stabilize harsh classroom shadows.
2. **Face Localization:** A lightweight face detector extracts bounding box coordinates (x, y, w, h) , cropping out facial regions and ignoring background classroom clutter.
3. **Liveness Gatekeeper:** The cropped image passes through a texture and reflection classifier to inspect moiré patterns and artificial screen borders. If confidence falls below our liveness cutoff, the crop is dropped immediately.
4. **Embedding Generation & Distance Matching:** Verified live faces are converted into a numeric vector array via an embedding model. We calculate the cosine distance between this vector and enrolled student profiles. If the distance value clears the threshold, the student's ID is retrieved.
5. **Deduplication & Record Storage:** Before committing an entry, the script verifies an active class session and checks if the student's ID has already been logged. If already present, the hit is ignored; otherwise, a new row is appended with the timestamp.
6. **Teacher Portal:** An administrative dashboard allows faculty to start class sessions, monitor student detections as they happen, and export final attendance records to CSV or Excel.

Camera Stream → *OpenCV Detection* → *Liveness Gatekeeper*
→ *Embedding Extraction* → *Vector Matching* → *Duplicate Filter*
→ *Database Commit*

System Architecture (High Level Diagram)

The proposed Smart Attendance System will follow a layered architecture consisting of the user interface layer, camera/input layer, computer-vision layer, artificial-intelligence layer, attendance-processing layer, database layer, and administration/reporting layer. The architecture is designed so that each component has a specific responsibility while communicating with the next stage of the system.





Project Outcome / Deliverables

- D1. Enrollment Module:** Onboarding utility to extract student profile information, roll numbers and baseline facial vector profile embeddings.
- D2. Video Ingestion Pipeline:** OpenCV based feed handler to ingest video feed, scale down video frames and extract faces without skipping frames.
- D3. Anti-Spoofing Filter:** Presentation Attack Detection module to detect phone screens and photo prints.
- D4. Biometric Matching Engine:** Vector similarity engine to match live face embeddings with registered student IDs using distance metric.
- D5. Attendance Backend:** Logic service to validate active lecture sessions and filter duplicates.
- D6. Local Database:** Storage layer to keep track of student metadata, reference embeddings and attendance records per date.
- D7. Faculty Management Portal:** Web dashboard with session management capabilities, real-time detection tables and CSV/Excel exports.
- D8. Testing & Validation Report:** Benchmark analysis report including accuracy, false acceptance rate (FAR) and latency under different illumination levels.

Assumption

The proposed Smart Attendance System is developed based on several assumptions concerning the classroom environment, students, hardware, software, and system operation. These assumptions define the expected operating conditions and help establish the boundaries of the project.

First, it is assumed that a **camera will be available** in the classroom or at the location where attendance is conducted. The camera should provide sufficient image quality for detecting and recognizing faces. Extremely poor-quality cameras, heavily blurred images, or severely damaged camera hardware may reduce system performance.

Camera Capabilities: The system relies on a minimum 720p resolution camera recording at 20–30 FPS, positioned to capture students as they enter or remain seated.

Controlled Enrollment : Initial student vector registration requires direct, balanced lighting to generate reliable reference embeddings.

Environmental Constraints: Designed for normal indoor classroom lighting. Direct window backlighting, pitch darkness, or heavy face occlusions (such as opaque masks or sharp angles beyond 45°) will lower accuracy.

Attack Boundaries: Defense is targeted against 2D presentation attacks (printed images, tablet screens, looped videos). Specialized 3D silicone masks are outside the project's current operational scope.

Local Connectivity: The edge system requires a stable local network connection to the database to prevent uncommitted attendance records.

Biometric Privacy: Student facial data is stored as mathematical embedding vectors rather than retaining raw unencrypted face photos.

References

1. OpenCV. **OpenCV Documentation and Computer Vision Resources**. OpenCV.org. OpenCV is an open source computer vision library which enables developers to do operations like face detection, object detection, deep learning based computer vision operations on image/video.
2. Guo, G., & Zhang, N. **A Survey on Deep Learning Based Face Recognition**. Computer Vision and Image Understanding, 2019. Survey article discussing deep learning based face recognition techniques and challenges in the domain of face recognition like pose, illumination, expression and feature representations.
3. Chakraborty, S., & Das, D. **An Overview of Face Liveness Detection**. Article talking about the susceptibility of face recognition systems to spoofing attacks and the need for liveness detection.
4. Madi, M. khammari, M., & Larabi, M.-C. **An Overview of Face Anti-Spoofing Methods for Face Recognition Systems**. 2026. Review paper about presentation attack detection techniques and presentation attacks like printed photograph, replay and 3D mask attacks.
5. Suresh Kumar, S. et al. **Face Recognition using Deep Learning in an Attendance Tracking System**. 2026. Research article describing an attendance tracking system where face is detected and attendance is recorded using deep learning techniques.
6. Pawale, S. et al. **Face Recognition Based Attendance Using Artificial Intelligence, Computer Vision, Deep Learning, OpenCV, and Python**. 2026. This paper explains real-time facial image capturing, feature extraction, face recognition, and automatic attendance and timestamp recording.
7. **A Survey on AI-Based Smart Attendance Systems Using Face Recognition, Liveness Detection and Secure Biometric Data Management**. 2026. Face recognition, liveness detection, biometric security, and dashboard for an intelligent attendance management system are discussed here.